Lecture 8
Dimensionality Reduction — PCA, Kernel PCA & Nonlinear Visualisation

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

ehirst@unicamp.br

Setembro 2026

Overview

1. Why Reduce Dimension?
2. Linear: PCA
3. Nonlinear Embeddings
4. Geometric Data Analysis
5. Summary

Recap (Lecture 7): Cluster definitions vary; the metric sets the method.
Guiding question: *How do we visualise data living in* $\mathbb{R}^{100}$*, faithfully?*

Seeing High-Dimensional Data

The task

Find a projection map, $\Pi : \{x_i\} \subset \mathbb{R}^n \to \mathbb{R}^m$, $m \in \{2, 3\}$ (visualisation), or $m \approx$ intrinsic dimension (coordinates), preserving structure of the data.

What might "preserve structure" mean? Candidate invariants:

- **Variance** along directions — PCA;
- **Pairwise distances** (extrinsic or geodesic) — MDS, Isomap;
- **Local neighbourhoods** (who is near whom) — t-SNE, UMAP;
- **Topology** (components, loops) — persistence (Lecture 7's aside).

No-free-embedding principle

$\to$ No map to $\mathbb{R}^2$ preserves all of these!
$\to$ Every method chooses what to sacrifice; **know the choice before analysing the visualisation**.

Uses beyond pictures

Denoising (project out noise directions), decorrelation (better basis identifies redundancy), compression, intrinsic-dimension estimation, preprocessing for clustering.

PCA as an Eigenproblem

Principal Component Analysis

Centre the data; form the covariance:

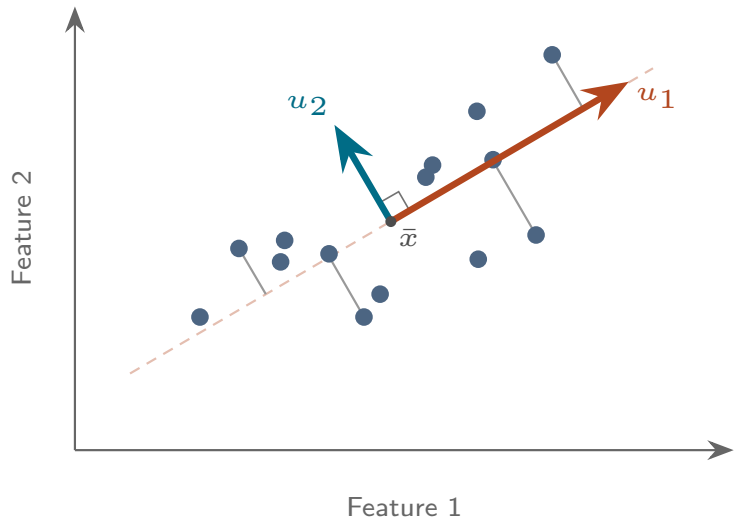
$$C = \frac{1}{N} X^\top X \in \mathbb{R}^{n \times n}.$$

The first principal direction maximises variance:

$$u_1 = \text{argmax}_{\|u\|=1}\, u^\top C u$$

...the top eigenvector of C (Rayleigh quotient); subsequent components: next eigenvectors, mutually orthogonal.

- Equivalent variational form: the rank-m projection minimising reconstruction error $\sum_i \|x_i - \Pi x_i\|^2$.
- Computed by **SVD** of X: stable, no explicit covariance needed.
- Captured variance $= \sum_{j \le m} \lambda_j / \sum_j \lambda_j$: an honest, quantitative quality measure — rare among embedding methods.



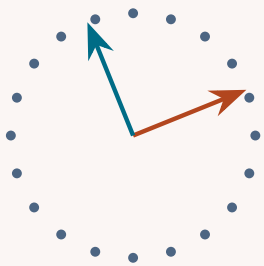
Principal directions of a 2D point cloud. Here u_1 captures 86% of the variance, $u_2 \perp u_1$ the rest; grey segments are the discarded residuals $x_i - \Pi x_i$.

PCA in Practice — and its Limits

- **Choosing m**: plot the eigenvalue spectrum; a sharp drop (spectral gap) suggests intrinsic linear dimension; slow decay warns that no small m is faithful.
- **Loadings are readable**: each component is an explicit linear combination of features — for feature vectors of invariants, components can express meaningful mathematical combinations (conjecture material).
- **Whitening**: rescale components to unit variance — useful preprocessing for methods assuming isotropy.
- **Always scale features first**: PCA maximises variance, so an ill-scaled feature hijacks the components (same warning as clustering, Lecture 7).

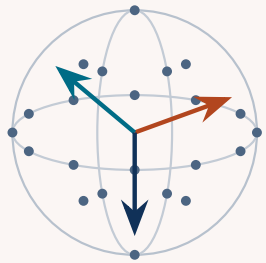
The fundamental limitation

PCA sees only *linear* structure: the best-fitting affine subspace. Curvature is invisible to it, and nonlinear methods exist precisely for this:



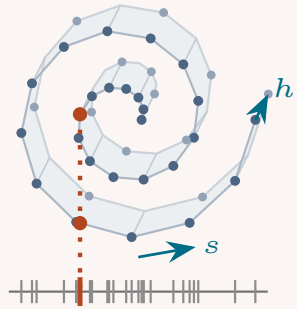
circle: $\lambda_1 = \lambda_2$

PCA 2, true 1



sphere: λ_i all equal

PCA 3, true 2



swiss roll: shadow folds

PCA 3, true 2

Kernel PCA: the Kernel Trick

Idea

→ Map data by a nonlinear $\phi : \mathbb{R}^n \to \mathcal{F}$ (possibly infinite-dimensional), do PCA *there*.
→ The trick: PCA in $\mathcal{F}$ needs only inner products, so a **kernel** $k(x, x') = \langle \phi(x), \phi(x') \rangle$ suffices, ... ϕ is never constructed.

The recipe:

- Choose a kernel: polynomial $k(x, x') = (x^\top x' + c)^d$ (features = monomials);
  Gaussian/RBF $k(x, x') = e^{-\|x - x'\|^2 / 2\sigma^2}$ (infinite-dimensional $\mathcal{F}$).
- Gram-matrix: compute $K_{ij} = k(x_i, x_j)$, double-centre it, compute the eigendecomposition (note K_{ij} is $N \times N$ instead of $n \times n$).
- Build new basis: Arrange N eigenvectors in decreasing eigenvalue as columns of a matrix (scaled by $\sqrt{\lambda}$). Rows are the projections, with $\leq \min(\dim(\mathcal{F}), N - 1)$ non-zero new coordinates.
- Full features: these can be reproduced as $v_j = \sum_i a_i^{(j)} (\phi(x_i) - \bar{\phi})$ for analysis but are not needed in this kernel trick.

...with this, nonlinear structure in $\mathbb{R}^n$ (concentric circles, curved varieties) can become *linearly* separable in $\mathcal{F}$ — Lecture 3's feature-map story, industrialised.

Example: $k(x, y) = (x^\top y)^2$ on $\mathbb{R}^2$

Feature $\phi(x) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2) \implies \langle \phi(x), \phi(y) \rangle = x_1^2 y_1^2 + 2 x_1 x_2 y_1 y_2 + x_2^2 y_2^2 = (x^\top y)^2$

Distance-Preserving: MDS and Isomap

Multidimensional scaling (MDS)

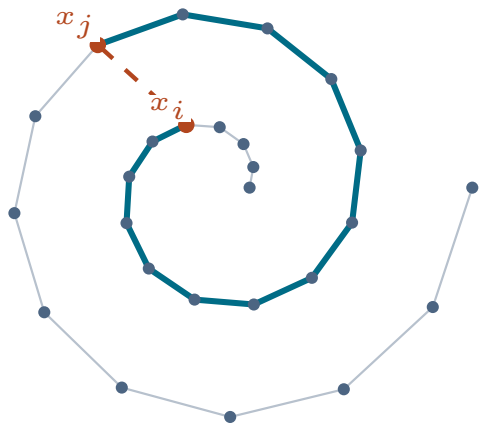
→ Given pairwise distances d_{ij}, find $y_i \in \mathbb{R}^m$ minimising $\sum_{i<j}(\|y_i - y_j\| - d_{ij})^2$ (stress).

→ Classical MDS on Euclidean distances = PCA (**dual view**):
...double-centring turns d_{ij}^2 into the Gram matrix $XX^\top$, which shares its non-zero spectrum with the covariance $X^\top X$ — one eigenproblem ($N \times N$ or $n \times n$).

Isomap

MDS on *geodesic* distances:

1. kNN graph on the data.
2. Shortest-path distances in the graph ≈ geodesic distances on $\mathcal{M}$;.
3. Classical MDS on these distances.



A spiral sampled of 26 points, consecutive samples joined. The dashed chord ≈ 0.7 cuts through space the data never occupies; the graph path (teal) ≈ 4.9 stays on the sheet.

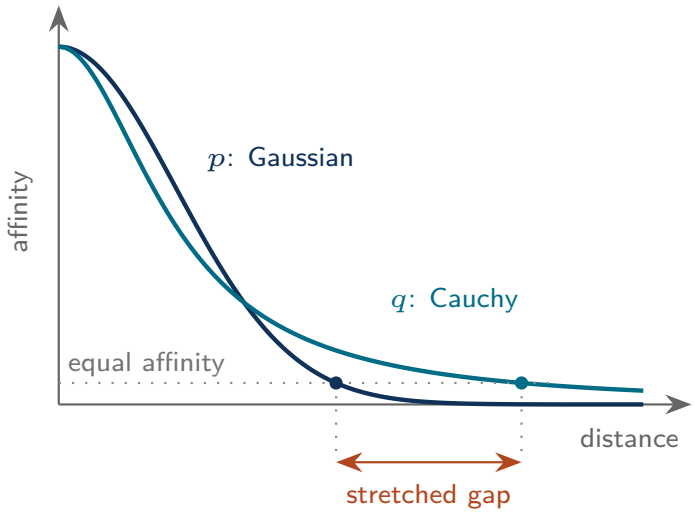
- Unrolls the swiss roll: geodesic distance along the sheet, not chordal distance through the ambient space.
- Guarantees for well-sampled convex-boundary manifolds; fails at holes, non-uniform sampling, and requires sampling dense enough that the graph metric converges.

Neighbourhood-Preserving: t-SNE

The t-SNE method

→ Match probability distributions of pairwise distances in high-dim space and low-dim reduction.
High-dim affinities: $p_{ij} \propto \exp(-\|x_i - x_j\|^2/2\sigma_i^2)$, Gaussian (per-point σ_i, symmetrised).
Low-dim affinities: $q_{ij} \propto (1 + \|y_i - y_j\|^2)^{-1}$, Student's t-dist (heavy tailed).
→ Minimise $\mathrm{KL}(P \,\|\, Q)$ over positions y_i by gradient descent.

- Asymmetric KL: cares strongly about keeping near neighbours near; barely about far pairs
  ⇒ low-dim good locally, unreliable globally.
- The heavy tail in q fights crowding: moderate high-dim distances are pushed far apart in low-dim
  ⇒ clusters separate crisply (but artificially).



→ High affinity = Points close
→ When close in high-dim then close in low-dim.
→ When far in high-dim then *even further* in low-dim.
→ Gaps are made here.

Note: Non-convex optimisation: different seeds give different pictures; orientation/arrangement meaningless.

Perplexity

→ Perplexity ($\sim$ 5–50) is a scale choice (try several).
→ Effectively the number of nearest neighbours:
perplexity$_i = 2^{H_i}$, for $H_i = -\sum_j p_{ij} \log_2 p_{ij}$
...solve for σ_i for all i, with root-finding.
→ Small ⇒ fine fragments, Large ⇒ merged blobs.

UMAP, and How to Read Any Embedding

UMAP

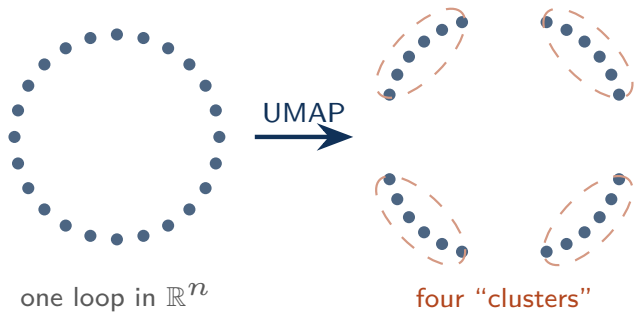
Parameters `n_neighbors` and `min_dist`.
Same skeleton as t-SNE, with modified distributions:

$$p_{ij} \propto \exp(-\max(0, d_{ij} - \rho_i)/\sigma_i)$$

...linear in distance, ρ_i dist to `n_neighbors`'th NN.

$$q_{ij} = \left(1 + a\,\|y_i - y_j\|^{2b}\right)^{-1}$$

...params (a, b) fitted to `min_dist`.
$\rightarrow$ In practice: faster, scales better,
preserves somewhat more global structure.



Uniform samples of a single loop — no clusters, no gaps.
An embedding can tear the loop and the fragments then read as four well-separated groups. Both the tearing and the grouping are produced by the algorithm $\rightarrow$ *caution*!

Safety rules

t-SNE/UMAP are very non-linear and unphysical, be cautious of:

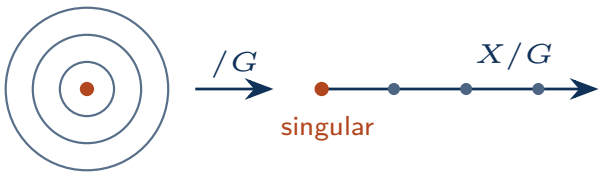
- Cluster sizes carry no meaning (expansion/contraction is local and uneven).
- Distances between clusters carry (almost) no meaning.
- Apparent clusters can be created by the algorithm on unclustered data (c.f. image)
  $\Rightarrow$ check with several perplexities/seeds.
- Continuous structures (curves, orbits) can be torn into fragments (c.f. image).

...therefore use them to *generate* hypotheses, never as *evidence* on their own.

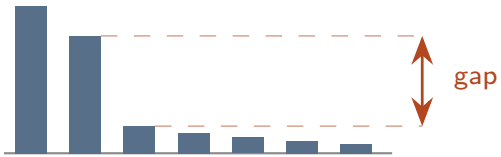
Applications in Geometry

- **Databases of geometries**: embed feature vectors of reflexive polytopes / CY hypersurfaces / graph ensembles; PCA spectra and UMAP clusters have repeatedly flagged families and outliers later understood theoretically (ML-for-landscape literature).
- **Symmetry orbits**: a G-orbit embeds as a low-dimensional structure; embeddings visually reveal orbit stratification (sharp changes in some property can indicate new strata).
- **Intrinsic dimension estimation**: gapped PCA spectra denotes an estimated lower intrinsic dimension $\Rightarrow$ a sanity check on the manifold hypothesis for your dataset (Lecture 2).

databases: families & outliers



orbits: strata of the quotient



spectrum gap: intrinsic dimension

Tutorial 8

→ PCA and kernel PCA on samples of embedded manifolds (sphere, torus, products).
→ t-SNE/UMAP on the same data across perplexities.
→ Catalogue which intrinsic/extrinsic properties each method preserved & destroyed, against truth.

Practice: Code and Checklist

Example (The standard pipeline)

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA, KernelPCA
from sklearn.manifold import TSNE, Isomap
import umap

Xs  = StandardScaler().fit_transform(X)   # every method uses distances
pca = PCA().fit(Xs)                       # keep the FULL spectrum
Z_pca = pca.transform(Xs)[:, :2]

Z_kpc = KernelPCA(n_components=2, kernel="rbf").fit_transform(Xs)
Z_iso = Isomap(n_neighbors=10, n_components=2).fit_transform(Xs)
Z_tsn = TSNE(perplexity=30, init="pca", random_state=0).fit_transform(Xs)
Z_ump = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=0).fit_transform(Xs)
```

Checklist for any embedding figure destined for a paper/talk:

- Features scaled? Method, every parameter, and seed reported?
- PCA spectrum shown — how linear is the data really?
- Several perplexities tried? Only *stable* structure claimed?
- Clusters correlated with a variable *not* used to build the embedding?
- Cluster sizes and gaps left uninterpreted?
- Every claim re-checked in the *original* space?

Summary

Key points

- Every embedding preserves some structure and destroys the rest; name the invariant before trusting the picture.
- PCA is an exact eigenproblem: best affine approximation, interpretable eigenspectrum — but blind to nonlinearity.
- Kernel PCA lifts data to non-linear features via inner products alone, linear in feature space. Kernel trick makes computation feasible.
- MDS matches a given distance matrix between a high-dim dataspace and a low-dim representation space. Isomap uses *geodesic* distances from a k-NN graph.
- t-SNE and UMAP preserve neighbourhoods, by matching distributions with KL.
- Cluster sizes and gaps are artefacts of the low-dim kernel, believe only what survives a change of seed and scale. Hypotheses, never evidence.
- Geometric applicaition: families and outliers in databases, strata of symmetry orbits, intrinsic dimension from a gapped spectrum.

Next

Tutorial 8: PCA, kernel PCA, t-SNE, UMAP on manifold samples.
Lecture 9: Reinforcement learning – learning by acting, for search problems in geometry.